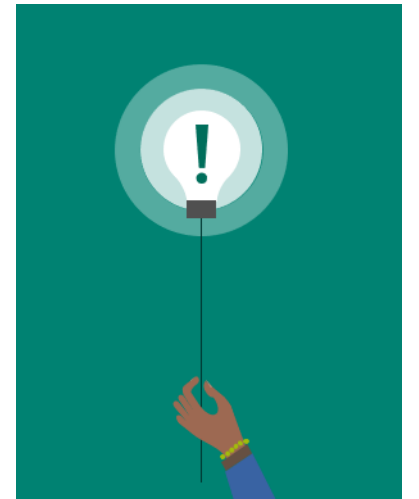


Developing Advanced Functions

Objectives

After completing this learning, you will be able to:

- Understand basic functions
- Understand basic parameters
- Understand the CmdletBinding attribute
- Understand how to use CmdletBinding properties
- Learn to return objects from functions
- Understand functions and the pipeline
- Review Process block
- Understand dynamic parameters
- Understand how to implement and use dynamic parameters



What is a Function?

Reusable code

Reduces size of code by reusing repeatable lines and thus increases reliability

Can accept parameter values and return output

Advanced functions behave like cmdlets

Should include help content for use with Get-Help (like cmdlets)

What is the difference between a script and a function?

Syntax

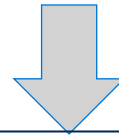
Param() statement is optional

```
Function [Scope:]<name>
{
    Param ($parameter1,$parameterN)
    <statement list>
    <statement list>
}
```

Creating a Utility Function

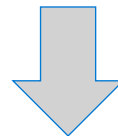
Long command(s) can be turned into a function to simplify future use

```
PS C:\> Get-Service -Name spooler -RequiredServices `
-ComputerName DC
```



```
Function Get-ServiceInfo {
    Get-Service -Name spooler -RequiredServices -ComputerName DC
}
```

Use function name to run the long command

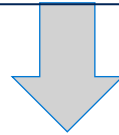


```
PS C:\> Get-ServiceInfo
```

Creating a Utility Function with Parameters

Turn cmdlet parameter values into function parameters to make code more dynamic

```
Function Get-ServiceInfo {  
    Param ($svc, $computer)  
    Get-Service -Name $svc -RequiredServices -ComputerName $computer  
}
```



```
PS C:\> Get-ServiceInfo -svc spooler -computer localhost
```

Questions?



Parameter Overview

- Function/script parameters can be named, positional, or dynamic
- Can be defined in three ways:
 - Param() keyword in body
 - Following function name before body
 - \$args automatic variable (positional only)

Parameters – No Param() statement

- Can appear in parenthesis () prior to the function body
- [CmdletBinding()] attribute not available
 - Discussed later

```
PS C:\> function <Name> ($p1,$p2,$pn)
{
    <PowerShell code>
}
```

Parameters – Param() statement

- Full set of advanced function parameter features available when using Param() within body

```
function <Name> {  
    param ($p1,$p2,$pn)  
}
```

- Set default values

```
function <Name> {  
    param ($p1 = "value1", $p2 = "value2", $pn = "valueN")  
}
```

- Strongly typed parameters

```
function <Name> {  
    param ([int]$p1 = "42", [string]$p2 = "value2")  
}
```

Parameter Binding

- PowerShell passes undefined parameters to \$args
 - Includes both parameter name and argument value
- \$PSBoundParameters returns hash table containing bound values to known parameters



Parameters – Unnamed, positional

- Automatic variable - \$args
- Array of undeclared parameter values passed to function, script, or script block
- Least-preferred technique

```
function Use-Args {  
    Write-Output "$($args[0]), $($args[1])"  
}  
Use-Args Hello world
```



Positional Arguments

- Great for simple functions
- Automatically implemented
- Order matters
- No built-in type validation
- Can be confusing with many arguments



```
PS C:\> Get-SystemUptime sv-001
```

```
PS C:\> Add-Numbers 45 76 89 23 74
```

```
PS C:\> New-DataRecord jdoe John Doe NJ 'Red Bank' US
```

Named Parameters

- Implement with the function declaration
- Implement with the Param() block
- Order doesn't matter
- Many validation options
- Additional parameters in \$args

```
PS C:\> Get-SystemUptime -Computername SV-001
```

```
PS C:\> Add-Numbers -Values 45,76,89,23,74
```

```
PS C:\> New-DataRecord -Username jdoe -Givenname John `
    -Surname Doe NJ -City 'Red Bank' US
```

Questions?



Switch Parameter

- Parameters with no parameter value
- To create a switch parameter in a function use the [Switch] type
- False by default, True if present
- Syntax

```
function SwitchExample {  
    Param([switch]$LightSwitch)  
    if ($LightSwitch) {  
        Write-Host "Power is on"  
    } else {  
        Write-Host "Power is off"  
    }  
}
```


Switch Parameter (Continued)

Example

```
function SwitchExample {  
    Param([switch]$LightSwitch)  
    if ($LightSwitch) {Write-Host "Power is on"} else  
        {Write-Host "Power is off"}  
}
```



```
PS C:\> SwitchExample  
Power is off
```

```
PS C:\> SwitchExample -LightSwitch  
Power is On
```

```
PS C:\> SwitchExample -LightSwitch:$false  
Power is off
```

```
PS C:\> SwitchExample -LightSwitch:$true  
Power is on
```

Demonstration

Parameter Basics



Questions?



[CmdletBinding()] Attribute

- Parameter binding now checks for valid parameters
 - Will fail on unknown parameters
- Provides access to certain functionality within \$PSCmdlet variable
- Automatically enables common parameters
- Optionally turn off positional binding
- Allows specifying default parameter set for ambiguous situations

```
Function New-EmployeeFile
{
    [cmdletbinding(...)]
    Param ($parameter1,$parameterN)
}
```

[CmdletBinding()] Attribute - Risk Mitigation

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>,
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

Activates support for Risk Mitigation when set to \$True

–Whatif and –Confirm common parameter now present



[CmdletBinding()] Attribute - Risk Mitigation cont.

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>,
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

- Sets general severity level of changes made by function
- Used in conjunction with \$ConfirmPreference automatic variable to control confirmation behavior



[CmdletBinding()] - DefaultParameterSetName

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>,
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

Specifies the parameter set to use when it cannot be determined from user input



[CmdletBinding()] - HelpURI

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>,
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

- Specifies an online version of a help topic that describes the function
- Must begin with "http" or "https"



[CmdletBinding()] - SupportsPaging

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>,
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

- Automatically adds First, Skip, and IncludeTotalCount parameters
- Allow users to select output from a large result set
- Designed for functions that return data from data stores, such as a SQL database



[CmdletBinding()] - PositionalBinding

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>,
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

- Determines whether function parameters are positional by default
- Default value is \$True when not present in CmdletBinding
- When parameters are positional, the parameter name is optional
- Position argument of the Parameter attribute takes precedence over the PositionalBinding argument



Demonstration

CmdeltBinding



Questions?



Begin / Process / End

- All script blocks can use statement lists
 - **Begin** – executed at beginning of pipeline command
 - **Process** – executed once for each pipeline object
 - **End** – executed once at end of pipeline command
- All functions in pipeline can have their own Begin, Process, and End blocks



```
function Add-Numbers
{
    begin { $total = 0 }
    process { $total += $_ }
    end { return $total }
}
```

```
PS C:\> 1..10 | Add-Numbers
55
```

Named blocks example

```
function My-Function
{
    Begin
    {
        Remove-Item .\Events.txt
        Write-Host "Start" -ForegroundColor Red
    }
    Process
    {
        $_.Message | Out-File -Filepath Events.txt -Append
    }
    End
    {
        Write-Host "End" -ForegroundColor Green
        notepad.exe Events.txt
    }
}
```

```
PS C:\> Get-EventLog -LogName Application -Newest 5 | My-Function
```

Pipeline Input

\$_

- Automatically available within each process block only
- Direct access to the current item in the pipeline
- No special access required



\$input

- Collection of current pipeline data
- Automatically available within Process and End blocks
 - Contains single item in Process block
 - Contains entire collection of data in End block
- `$input.MoveNext()` and `$input.Current`

Demonstration

Pipeline



Questions?



Developing Advanced Functions

45 minutes

